COMP 2402 AB - Fall 2021
Assignment #2

Due: Sunday October 10, 23:59

Submit early and often. No late submissions will be accepted.

Academic Integrity

You may:

- Discuss general approaches with course staff,
- Discuss general approaches with your classmates,
- Use code and/or ideas from the textbook,
- Use a search engine / the internet to look up basic Java syntax,
- Send your code / screen share your code with course staff (although it is unlikely the course staff has the bandwidth to look at individuals' code, unfortunately.)

You may not:

- Send or otherwise share code or code snippets with classmates,
- Use code not written by you, unless it is code from the textbook (and you should cite it in comments),
- Use a search engine / the internet to look up approaches to the assignment,
- Use code from previous iterations of the course, unless it was solely written by you,
- Use the internet to find source code or videos that give solutions to the assignment.

If you ever have any questions about what is or is not allowable regarding academic integrity, please do not hesitate to reach out to course staff. We will be happy to answer. Sometimes it is difficult to determine the exact line, but if you cross it the punishment is severe and out of our hands. Any student caught violating academic integrity, whether intentionally or not, will be reported to the Associate Dean and be penalized as follows:

- First offence: F in the course.
- Second offence: One-year suspension from program.
- Third offence: Expulsion from the University.

These are standard penalties. More-severe penalties will be applied in cases of egregious offences. For more information, please see Carleton University's Academic Integrity page.

Grading

This assignment will be tested and graded by a computer program (and **you can submit as many times as you like, your highest grade is recorded**). For this to work, there are some important rules you must follow:

- Keep the directory structure of the provided zip file. If you find a file in the subdirectory `comp2402a2` leave it there.
- Keep the package structure of the provided zip file. If you find a package `comp2402a2;` directive at the top of a file, leave it there.
- Do not rename or change the visibility of any methods already present. If a method or class is public leave it that way.
- Do not change the `main(String[])` method of any class. Some of these are setup to read command line arguments and/or open input and output files. Don't change this behaviour. Instead, learn how to use command-line arguments to do your own testing.
- Submit early and often. The submission server compiles and runs your code and gives you a mark. You can submit as often as you like and only your latest submission will count. There is no excuse for submitting code that does not compile or does not pass tests.
- Write efficient code. The submission server places a limit on how much time it will spend executing your code, even on inputs with a million lines. For some questions it also places a limit on how much memory your code can use. If you choose and use your data structures correctly, your code will easily execute within the time limit. Choose the wrong data structure, or use it the wrong way, and your code will be too slow for the submission server to grade (resulting in a grade of 0).

Submitting and Server Tests

The submission server is ready: here. If you have issues, please post to Discord to the teaching team (or the class) and we'll see if we can help.

When you submit your code, the server runs tests on your code. These are bigger and better tests than the small number of tests provided in the "Local Tests" section further down this document. They are obfuscated from you, because you should try to find exhaustive tests of your own code. This can be frustrating because you are still learning to think critically about your own code, to figure out where its weaknesses are. But have patience with yourself and make sure you read the question carefully to understand its edge cases.

Warning: Do not wait until the last minute to submit your assignment. There is a hard 5 second limit on the time each test has to complete. If the server is heavily loaded, borderline tests may start to fail. **You can submit multiple times and your best score is recorded.**

Changes from Assignment 1's Server:

- The java command being run is shown to you for each test (modulo irrelevant-to-you stuff)
- The points for each test is shown to you for each test (+/- depending on pass/fail)

- If you fail an early test, the remaining tests get run so you should get more partial credit. However, if I find that this breaks the server too much (aborting on failure prevents damaging programs from doing more damage), I may have to turn this feature off. With great power comes great responsibility, so test your programs locally before submitting to the server!
- If your output differs from what the server expects, the error message at least tells you what you got vs. what was expected (i.e. now it's labelled so you don't have to guess which was which.)

The Assignment

Purpose & Goals

Generally, the assignments in this course are meant to give you the opportunity to practice with the topics of this course in a way that is challenging yet also manageable. At times you may struggle and at others it may seem more straight-forward; just remember to keep trying and practicing, and over time you will improve.

Specifically, assignment 2 aims to improve your understanding of the array-based data structures we've been talking about by:

- Using the ArrayList, ArrayQueue, and/or ArrayDeque interfaces to solve a variety of problems, recognizing when one may be more appropriate than another, and
- Implementing our own special Stack and Deque-like interfaces using some of the concepts we've been learning about.

Part A: The Setup

Start by downloading and decompressing the Assignment 2 Zip File (comp2402a2.zip), which contains a skeleton of the code you need to write. You should have the files: Part0.java, Part1.java, Part2.java, Part3.java, MyStack.java, MyFastStack.java, MyDeque.java, MyFastDeque.java and Tester.java.

As with assignment 1, you can test Parts0-3 using the command line or input/output files. You can test Parts 4 and 5 with the Tester class included in the zip file. As before, you'll want to add your own tests as this is just given as a basis on which you should build. The submission server will do thorough testing; it is your job to read the specifications as carefully as possible, and to examine your own code for potential problems and test those cases.

Part B: The Interface

As with assignment 1, the file `Part0.java` in the zip file actually does something (although it's a different something than assignment 1). You can use its `doIt(x)` method as a starting point for your solutions. Compile it. Run it. Test out the various ways of inputting and outputting data. You should not need to modify `Part0.java`, it is a template.

You can download some sample input and output files for each question as a zip file (**a2-io.zip**). If you find that a particular question is unclear, you can possibly clarify it by looking at the sample files for that question. Note that these are very limited tests; you can and should add to them to test your code more thoroughly as the server tests are much more extensive.

1. [10 marks] Read the input one line at a time and output every `x`'th line, starting with line 0, where `x>0` is a parameter to the `doIt` method. To get full points, try to use as little extra space and time as possible.
2. [10 marks] Read the input one line at a time and output every `x`'th line from the end, starting with the last line, where `x>0` is a parameter to the `doIt` method. To get full points, try to use as little extra space and time as possible.
3. [10 marks] Read the input one line at a time, and output the last `x` lines in reverse order, where `x>=0` is a parameter to the `doIt` method. If there are fewer than `x` lines, print the `n < x` existing lines in reverse. (The lines themselves are forwards, their order is reversed. See the sample input/output to clarify if you need it.) To get full points, try to use as little extra space and time as possible.

Part C: The Implementation

Files Provided

The starter code contains `MyStack` and `MyDeque` interfaces, which are modified `Stack` and `Deque` interfaces described below. Part C involves you completing implementations of these interfaces, `MyFastStack` and `MyFastDeque`, in the files `MyFastStack.java` and `MyFastDeque.java`, respectively. These are described in more detail in Parts 4 and 5 below.

Interface Semantics

Both `MyStack` and `MyDeque` represent a sequence of items where any neighbouring duplicates have (both) been removed. For example, the original sequence `[a,b,c,c,b,d]` would shrink to just `[a,d]` (since once the neighbouring `c,c` are removed, `b` and `b` become neighbouring duplicates and are also removed.)

As another example, the sequence `[a,b,c,c,c,b,d]` would shrink to `[a,b,c,b,d]` since once the first neighbours `c,c` are removed, a third `c` remains.

The individual elements are atomic (i.e. indivisible), so `[ab,ab]` shrinks to `[]`, and `[ba,ab]` does not shrink at all.

Finally, if you have the sequence `[a,b,c,c,b,a]`, this would shrink to the empty sequence `[]`.

Your Tasks

4. [20 marks] For the `MyStack` interface, we only wish to access the most recently added element in this "shrunken" sequence (the sequence with pairs of duplicates removed). That is, we want 3 methods:
    - `push(x)` - add **non-null** `x` to the sequence

- `pop()` - remove the most recent non-consecutive duplicate (null if no such element exists)
- `size()` - return the length of the shrunken sequence

For example, starting from an empty `MyStack`, the operations

- After `push(a),push(b),push(b)`
  - `size()` would return `1` and `pop()` would return `a` because the constructed sequence `[a,b,b]` shrinks to `[a]`
- After `push(a),push(b),push(b),push(b)`
  - `size()` would return `2` and `pop()` would return `b` because the constructed sequence `[a,b,b,b]` shrinks to `[a,b]`. Another `pop()` would return `a`.
- After `push(a),push(a)`
  - `size()` would return `0` and `pop()` would return `null`, since no element exists
- After `push(a), push(b), push(c), push(c), push(b)`
  - `size()` would return `1` and `pop()` would return `a` because the constructed sequence `[a,b,c,c,b]` shrinks to `[a]`.

Your task is to implement the `push(x)`, `pop()`, and `size()` methods in `MyFastStack` such that all three operations run in O(1) time. You may use JCF ADTs such as ArrayLists in your solutions.

5. [25 marks] For the `MyDeque` interface, we wish to access the shrunken sequence from either end (for both adding and removing). That is, we want the 5 methods:

   - `addFirst(x)` - add **non-null** `x` to the beginning of the sequence
   - `addLast(x)` - add **non-null** `x` to the end of the sequence
   - `removeLast()` - remove the "right-most" non-consecutive duplicate (null if no such element exists)
   - `removeFirst()` - remove the "left-most" non-consecutive duplicate (null if no such element exists)
   - `size()` - return the length of the shrunken sequence

   For example, starting from an empty `MyDeque`, the operations

   - After `addFirst(a),addFirst(b),addFirst(b)`
     - `size()` would return `1` and `removeLast()` would return `a` because the constructed sequence `[b,b,a]` shrinks to `[a]`
   - After `addLast(b),addFirst(b),addLast(b),addFirst(a)`

  - `size()` would return `2` and `removeFirst()` would return `a` because the constructed sequence `[a,b,b,b]` shrinks to `[a,b]`. Another `removeFirst()` would return `b`.

- After `addFirst(a),addLast(a)`
  - `size()` would return `0` and `removeFirst()` would return `null`, since no element exists

Your task for this question is to implement all 5 operations in O(1) time per operation in `MyFastDeque`. You may use JCF ADTs such as ArrayLists in your solutions.

Local Tests

For Parts 1, 2, and 3, you can download some sample input and output files for each question as a zip file (a2-io.zip). Once you get a sense for how to test your code with these input files, write your own tests that test your program more thoroughly (**the provided tests are not exhaustive!**)

For Parts 4 and 5 (`MyFastStack`, `MyFastDeque`), a `Tester` class is provided with some tests of the provided methods. I recommend you add tests to `Tester.java` to test your `MyFastStack` and `MyFastDeque` locally (as opposed to on the server).

Testing suggestions:

- Test incrementally, that is, bit-by-bit,
- Test edge cases, e.g. empty files, empty lists, parameters of odd or even sizes,
- Use small test cases at first, then go bigger only when necessary,
- Test individual methods as you write them, even. As in: you can test an incomplete method to make sure it's doing what you're expected thus far.

As an example of how to run tests (in this case, `Part0` and `Tester`) from the command line:

```
student@comp2402:a2 % ls
a1-io      a2-io      comp2402a1 comp2402a2 outfile1
a1-io.zip  a2-io.zip  comp2402a1.zip  comp2402a2.zip
student@comp2402:a2 % javac comp2402a2/*.java
student@comp2402:a2 % java comp2402a2/Part0 3
a
b
c
a
b
c
Execution time: 1.4200235780000001
student@comp2402:a2 % java comp2402a2/Part0 3 a2-io/part1a.in
```

```
apple
orange
banana
Execution time: 0.0010258700000000001
student@comp2402:a2 % java comp2402a2/Part0 4 a2-io/part1a.in
apple
orange
banana
banana
Execution time: 4.6180200000000003E-4
student@comp2402:a2 % java comp2402a2/Part0 4 a2-io/part1a.in out.txt
Execution time: 4.90298E-4
student@comp2402:a2 % cat out.txt
apple
orange
banana
banana
student@comp2402:a2 % java comp2402a2/Tester
Test myStack------
push(1)
comp2402a2.MyFastStack@5a07e868
push(7)
comp2402a2.MyFastStack@5a07e868
push(14)
comp2402a2.MyFastStack@5a07e868
push(11)
comp2402a2.MyFastStack@5a07e868
push(10)
comp2402a2.MyFastStack@5a07e868
push(12)
comp2402a2.MyFastStack@5a07e868
push(7)
comp2402a2.MyFastStack@5a07e868
push(14)
comp2402a2.MyFastStack@5a07e868
push(6)
comp2402a2.MyFastStack@5a07e868
push(2)
comp2402a2.MyFastStack@5a07e868
Done Test myStack------
Test myDeque------
addFirst(13)
comp2402a2.MyFastDeque@3fee733d
```

```
addFirst(6)
comp2402a2.MyFastDeque@3fee733d
addLast(0)
comp2402a2.MyFastDeque@3fee733d
addLast(14)
comp2402a2.MyFastDeque@3fee733d
addFirst(7)
comp2402a2.MyFastDeque@3fee733d
addFirst(5)
comp2402a2.MyFastDeque@3fee733d
addLast(1)
comp2402a2.MyFastDeque@3fee733d
addLast(9)
comp2402a2.MyFastDeque@3fee733d
addFirst(12)
comp2402a2.MyFastDeque@3fee733d
addFirst(9)
comp2402a2.MyFastDeque@3fee733d
Done Test myDeque------
student@comp2402:a2 %
```

Tips, Tricks, and FAQs

How should I approach each problem?

- Make sure you understand it. Construct small examples, and compute (by hand) the expected output. If you aren't sure what the output should be, go no further until you get clarification.
- Now that you understand what you are supposed to output, and you've been able to solve it by hand, think about how you solved it and whether you could explain it to someone. How about explaining it to a computer?
- If it still seems challenging, what about a simpler case? Can you solve a similar or simplified problem? Maybe a special case? If you were allowed to make certain assumptions, could you do it then? Try constructing your code incrementally, solving the smaller or simpler problems, then, only expanding scope once you're sure your simplified problems are solved.

How should I test my code?

- You should be testing your code as you go along.
- Start small so that you can compute the correct solution by hand.
- Think about edge cases.
- Think about large inputs, random inputs.
- Test for speed.